CX651 — Assignment 8

JVM Observation Lab: Bytecode and Garbage Collection

Overview

In this assignment, you will look inside the JVM — not by writing `new` code, but by examining what the compiler produces. You will compile a provided class, read its bytecode using `javap`, trace a method call through the operand stack, and explain what each instruction does. You will then run a second program that stresses the heap, observe the garbage-collection log, and explain what the log reveals about how the JVM manages memory.

General Instructions

You will run several experiments and record your answers directly in this document, then submit it to Gradescope. The following files are provided:

- `IntMath.java` — an arithmetic class with three `static` methods. Compile it and examine its bytecode.
- `HeapStress.java` — a program that allocates many short-lived objects to trigger garbage collection. Run it with GC logging enabled and interpret the output.

Setup

Verify Java 21 or later is installed:

```
java -version   # must print 21 or higher

javac -version
```

Run all commands below from the directory containing the `.java` files.

Part A — Bytecode Reading

Step 1: Compile.

```
javac IntMath.java
```

Step 2: Disassemble.

```
javap -c IntMath
```

https://docs.oracle.com/javase/specs/jvms/se21/html/jvms-6.html Keep the bytecode output visible as you answer the questions. Refer to the JVM Instruction Reference at as needed.

The `add` method

Q1. What is the first instruction in `add`? Where does its operand come from, and what does it push onto the stack?

Answer:

public static int add(int, int);

Code:

0: iload_0

1: iload_1

2: iadd

3: ireturn

The first instruction in method add is iload_0.

The 0th-index element comes from the local variable array of the current (add method's) frame, which is also the method's 1st parameter.

The value of the local variable at index 0 is pushed onto the operand stack.

Q2. Trace the operand stack step by step through the entire `add` method. Write each step as: `[ ] → <instruction> → [stack contents, top on right]`.

Answer:

For add(3, 4)

Local variable array = [3, 4]

Offset - instruction - operand stack

[0] - iload_0 - [3]

[1] - iload_1 - [3, 4]

[2] - iadd - [7]

[3] - ireturn - [] : poped and placed on caller's operand stack.

Q3. What does `ireturn` do? Could the compiler use `lreturn` or `dreturn` here instead? Explain why or why not.

Answer:

ireturn returns an int value from the current frame/method to the caller method by popping the top element from the current operand stack and pushing it onto the invoker's operand stack. Control is shifted to the caller method frame.

lreturn and dreturn are used to return long and double values, respectively. Using them will result in a signature mismatch, as the 'add' method has an int return type, resulting in invalid bytecode.

The `factorial` method

Q4. Find the instruction that performs the recursive call `factorial(n-1)`. Write its mnemonic. What does the mnemonic tell you about how the JVM handles dispatch for `static` methods?

Answer:

invokestatic is used within the factorial method to invoke itself with a new frame each time.

JVM uses the reference to the static method from the constant pool to execute it.

Q5. Identify the branch instruction in the `factorial` method and its target offset. Under what condition does the JVM take the branch? What happens if it does not?

Answer:

"if_icmpgt " with target offset 7.

If the parameter "n" is greater than the constant '1', the JVM takes a branch to the instruction at offset 7, i.e., iload_0.

If the condition is not met, it pushes the constant value 1 onto the stack, and the next return instruction returns 1 to the calling method.

Q6. Count the `invokestatic` instructions in the `factorial` method. Examine the method's control flow. Did `javac` preserve the recursive call or convert the method into an iterative loop? Support your answer using the bytecode. What operation occurs immediately after the recursive call returns?

Answer:

1. There is only one "invokestatic" instruction.

2. The Java compiler preserves recursion because there is no loop instruction with a specified range. It just has condition-based invocation of the factorial method with n-1 param each time (using isub). Each recursive call creates a new stack frame. And the value of 'n' is stored in the 0th index of the local variable array of the current frame each time.

3. Code Reference:

0: iload_0 -> Each time, the value at the same index is loaded for comparison, i.e., the 1st param of the method.

1: iconst_1

2: if_icmpgt 7

…

11: invokestatic #7 -> invoke method creates a new frame with its own local variable list and operand stack.

4. At offset 14, Imul multiplies the returned factorial(stored at top of stack) by the original n(top - 1). It stores the result at the top of the operand stack, then it is returned to the caller frame.

Full annotation of the `square` method

Q7. Copy the complete bytecode for the `square` method and annotate every instruction. For each instruction provide: (a) its full name, (b) what it pops, (c) what it pushes. Use inline comments. Example format:

Annotation format:

```
<offset>: <instruction>  // Full name:
                         // Pops:
                         // Pushes:
```

Paste your annotated `square` bytecode here:

Answer:

public static int square(int);

```
  Code:
    0: iload_0   // Full name: Load int from local variable 0
                 // Pops: Nothing
                 // Pushes: The value of the parameter int onto operand stack

    1: iload_0   // Full name: Load int from local variable 0
                 // Pops: Nothing
                 // Pushes: The value of the parameter int

    2: imul      // Full name: Multiply two integers
                 // Pops: The top 2 int values from the operand stack
                 // Pushes: Their product (int)

    3: ireturn   // Full name: Return int from method
                 // Pops: The int result from the operand stack
                 // Pushes: Nothing on the current operand stack; pushes the value to the caller operand stack
```

Part B — Garbage-Collection Log Interpretation

Step 1: Compile `HeapStress.java`.

```
javac HeapStress.java
```

Step 2: Run with GC logging. The `-Xms` and `-Xmx` flags constrain the heap to make GC events easier to observe.

```
java -Xms16m -Xmx64m -Xlog:gc*:file=gc.log:time,uptime,level,tags HeapStress
```

Step 3: Open `gc.log` in a text editor. Examine the JVM initialization lines for Q8–Q9, then locate the garbage-collection events for the remaining questions. The first event is typically identified by `GC(0)`.

Q8. Find the JVM version/build information in the initialization section of `gc.log` and copy the complete line. Its wording may vary. For example, it may begin with `Version:`. If your log does not

include this information, paste the output of `java -version` instead. Include vendor information if it appears; you do not need to obtain it separately.

Note: Exact wording, event types, and numerical values may vary by platform, garbage collector, and JVM version. Base your answers on your own output rather than matching the examples exactly.

Answer:

JVM Version line: 21.0.12.1

```
[2026-10-22T18:29:46.075-0700][0.008s][info][gc,init] Version: 21.0.12.1+1-LTS (release)
```

Q9. Identify the garbage collector used in your run and copy the log line that identifies it. Give the collector's full name if the log uses an abbreviation. For example, `Using G1` identifies the Garbage-First (G1) collector.

Answer:

Default GC, Garbage First - G1 is being used.

```
[2026-10-22T18:29:46.075-0700][0.008s][info][gc      ] Using G1
```

Q10. Find the first garbage-collection event in `gc.log`. An event may span several lines; select its summary line, which shows heap usage before and after collection, heap capacity, and pause duration. Begin your answer with the exact uptime shown on that summary line, then copy the complete line. Identify: (a) heap in use before collection, (b) heap in use after collection, (c) total heap capacity at that event, and (d) pause duration.

Answer:

Uptime: 0.046s

```
[2026-10-22T18:29:46.113-0700][0.046s][info][gc           ]
GC(0) Pause Young (Normal) (G1 Evacuation Pause) 9M->1M(18M)
0.385ms
```

(a) 9 MB

(b) 1 MB

(c) 18 MB

(d) 0.385ms

Q11. Examine the first young-generation garbage-collection event. Describe the changes in Eden, survivor, and old-region counts, citing specific log lines. Explain what these changes suggest about reclaimed and surviving objects, and what they cannot tell you about individual objects.

Answer:

```
[2026-10-22T18:29:46.113-0700][0.046s][info][gc,heap      ]
GC(0) Eden regions: 8->0(8)
[2026-10-22T18:29:46.113-0700][0.046s][info][gc,heap      ]
GC(0) Survivor regions: 0->1(1)
[2026-10-22T18:29:46.113-0700][0.046s][info][gc,heap      ]
GC(0) Old regions: 2->2
```

Eden: Decreased from 8 regions to 0. The used Eden regions were emptied during collection.

Survivor: Increased from 0 to 1 region. Some objects survived collection and were moved into a survivor region.

Old: Remained at 2 regions.

These changes suggest that many objects in Eden were reclaimed, while some survived and were moved to the survivor region of the heap. However, region counts alone cannot tell us which individual objects survived or how many objects were reclaimed.

Q12. Phase 4 calls `.clear()` on both lists, removing their references to the retained objects. Does this guarantee that those objects will be garbage-collected before the program exits? Explain the difference between becoming eligible for garbage collection and having memory reclaimed. Refer to your log where relevant, but you do not need to identify the exact time Phase 4 occurred.

Answer:

No, calling .clear() on both lists removed their references, making those objects eligible for garbage collection. However, the JVM decides when to run garbage collection. The program can exit before the garbage collector reclaims those objects. The heap exit log reports a total of 58368k, used 32638k, but this alone does not establish whether those objects were reclaimed.

```
[2026-10-22T18:29:46.430-0700][0.364s][info][gc,heap,exit]
garbage-first heap   total 58368K, used 32638K
[0x00000007fc000000, 0x0000000800000000)
[2026-10-22T18:29:46.431-0700][0.364s][info][gc,heap,exit]
region size 1024K, 2 young (2048K), 1 survivors (1024K)
```

Q13. In 2–3 sentences, explain the generational hypothesis and why it makes GC efficient. Use at least one concrete number from your `gc.log` to support your answer.

Answer:

The generational hypothesis states that most objects become unreachable shortly after creation, while a smaller number survive for a longer time. The JVM improves efficiency by splitting the heap memory into young and old generations, collecting the young generation frequently instead of repeatedly scanning the entire heap.
In GC logs, the first GC reduced the heap ussage from 9M to 1M , ie 8MB in a pause of just 0.385ms.

Submission

Submit a single PDF named **A08_<BU-ID>.pdf**, replacing <BU-ID> with your BU ID.

Your PDF must include answers to **all questions, Q1–Q13**, including the annotated bytecode for Q7 and the requested output or log lines for Q8–Q10.

You may enter your answers directly in this document or use a separate document with each answer clearly labeled by question number.

When finished, save or export your document as a PDF:

- Microsoft Word on Windows: Select File → Save As, then choose PDF as the file type.

- Microsoft Word on Mac: Select File → Save As, then choose PDF from the File Format menu.
- Pages on Mac: Select File → Export To → PDF.
- Google Docs: Select File → Download → PDF Document (.pdf).

Open the PDF to check that all answers and code annotations are readable, then submit it through the Gradescope assignment link on Blackboard.